

بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-214 Computer Architecture and Organization

30 - Basics of Logic Design

Introduction

- Brief discussion of the basics of logic design
- To understand some of the motivation behind how computers are implemented, material will serve as a useful introduction
- Introduce basic building blocks of logic, namely, *gates*
- Use building blocks to construct *combinational* logic systems
- Design an ALU for MIPS processor
- Introduction to clocking
- Introduce memory elements

Digital Signals

- Electronics inside a modern computer are *digital*
- Digital electronics operate with only two voltage levels
 - high voltage and low voltage
- Computers are digital is a key reason they use binary numbers
- Rather than refer to voltage levels, we talk about signals that are logically
 - true, or 1, or are asserted or
 - false, or 0, or are deasserted
- Values 0&1 are called *complements* or *inverses* of one another

Gates

- Logic blocks are built from **gates** that implement basic logic functions
 - An **AND** gate implements AND function, and an **OR** gate implements OR function
 - Logical function **NOT** is implemented with an inverter that always has a single input
- Standard representation of these logic building blocks is shown



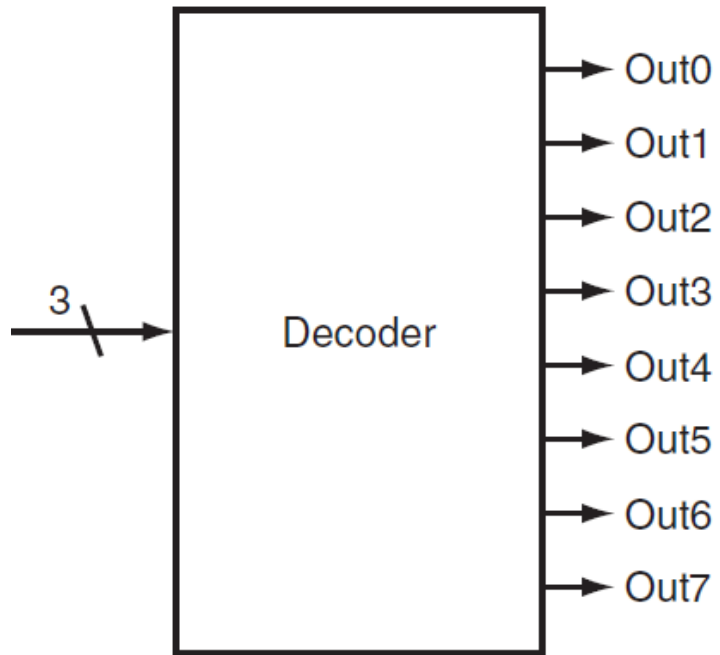
Combinational Logic

- We look at a couple of larger logic building blocks that we use heavily
 - Decoders
 - Multiplexors

Decoder

- Has n -bit input and 2^n outputs
- Only one output is asserted for each input combination
 - Outputs are usually numbered, say, Out0, Out1, ... , Out $2^n - 1$
- If value of input is i , then
 - Out i will be true
 - all other outputs will be false
- Figure shows a 3-bit decoder and truth table
 - Called a *3-to-8 decoder* since there are 3 inputs and 8 (2^3) outputs

Decoder



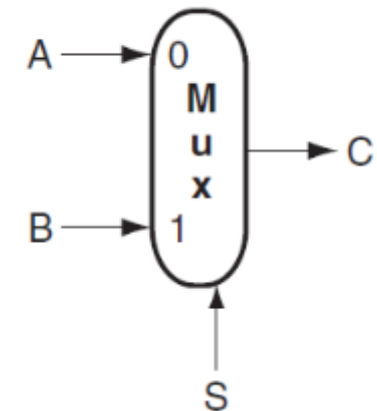
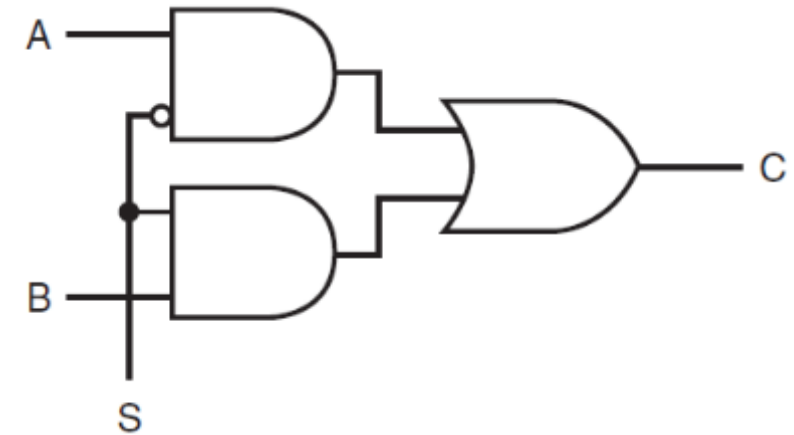
a. A 3-bit decoder

Inputs			Outputs							
12	11	10	Out7	Out6	Out5	Out4	Out3	Out2	Out1	Out0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

b. The truth table for a 3-bit decoder

Multiplexor

- Also called *selector*
 - since its output is one of inputs that is selected by a control
- Two-input multiplexor
 - has three inputs: two data values and a *selector* (control) value
 - *Selector* value determines which of the inputs becomes the output
 - Logic function: $C = (A \cdot S') + (B \cdot S)$



Don't Cares

- Often in implementing some combinational logic, there are situations where we do not care what value of some output is,
 - Either because another output is true or because a subset of input combinations determines values of outputs
- Such situations are referred to as *don't cares*
- Don't cares are important because they make it easier to optimize implementation of a logic function

Don't Cares

- Two types
 - *Output don't cares*: don't care about value of output for some input combination
 - Appear as *Xs* in the output portion of truth table
 - Designer or logic optimization program is free to make output *T* or *F* for that input combination
 - *Input don't cares*: an output depends on only some of the inputs
 - Shown as *Xs*, in input portion of truth table

Arrays of Logic Elements

- Many of the combinational operations to be performed on data have to be done to an entire word (32 bits) of data
- Often build an array of logic elements, which we can represent simply by showing that a given operation will happen to an entire collection of inputs

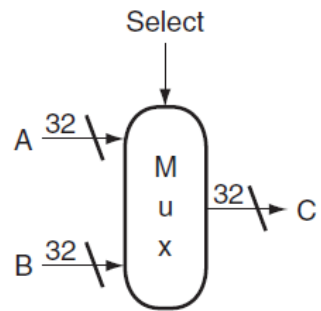
Bus

- **Bus** is a collection of data lines that is treated together as a single logical signal
- Indicate a signal is a bus rather than a single 1-bit line by showing it with a thicker line in a figure

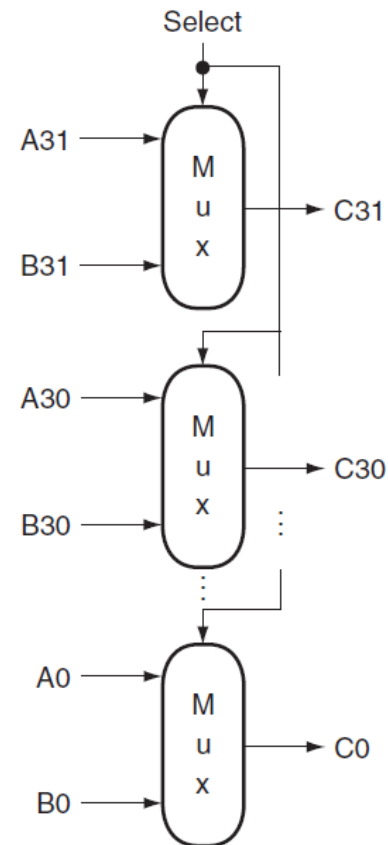
32-bit Buses

- Much of the time want to select between a pair of *buses*
- For example, result of an instruction that is written into a register can come from one of two sources
 - Multiplexor is used to choose which of the two buses (each 32 bits wide) will be written into the Result register
 - 1-bit multiplexor will need to be replicated 32 times
 - Figure shows how a multiplexor selects between pair of 32-bit buses

Array of Multiplexors



a. A 32-bit wide 2-to-1 multiplexor



b. The 32-bit wide multiplexor is actually an array of 32 1-bit multiplexors